

OSSP-PRACTICAL-WEEK8

TASK1:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Source code:

```
GNU nano 7.2                                memory.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc()
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("malloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    // calloc()
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);
}
```

```
printf("calloc: ");
for (i = 0; i < 5; i++)
    printf("%d ", b[i]);
printf("\n");

// realloc()
a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++)
    a[i] = i + 1;

printf("realloc: ");
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);
printf("\n");

// free()
free(a);
free(b);

printf("Memory freed successfully.\n");

return 0;
```

Output:

```
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ gcc memory.c -o memory
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./memory
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ valgrind --leak-check=full ./memory
==1162== Memcheck, a memory error detector
==1162== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1162== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1162== Command: ./memory
==1162==
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed successfully.
==1162==
==1162== HEAP SUMMARY:
==1162==      in use at exit: 0 bytes in 0 blocks
==1162==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==1162==
==1162== All heap blocks were freed -- no leaks are possible
==1162==
==1162== For lists of detected and suppressed errors, rerun with: -s
==1162== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

Observation:

I learned how dynamic memory allocation works using `malloc()`, `calloc()`, `realloc()`, and `free()`. I understood how memory can be allocated, resized, initialized, and released dynamically. I also learned how Valgrind can be used to detect memory leaks and check whether allocated memory is properly released.

Task2:

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (10 * 1024 * 1024)

void show_memory(const char *name)
{
    FILE *fp;
    char line[256];

    fp = fopen("/proc/self/status", "r");

    if (fp == NULL)
    {
        perror("fopen");
        return;
    }

    while (fgets(line, sizeof(line), fp))
    {
        if (strncmp(line, "VmRSS:", 6) == 0)
        {
            printf("%s %s", name, line);
            break;
        }
    }

    fclose(fp);
}

int main()
{
    char *data;
    pid_t pid;
```

```
}

int main()
{
    char *data;
    pid_t pid;

    data = malloc(SIZE);

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
        data[i] = 'A';

    printf("Parent PID: %d\n", getpid());
    show_memory("Parent before fork:");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild PID: %d\n", getpid());

        show_memory("Child before modification:");

        printf("Child modifying data...\n");
    }
}
```

```
    free(data);
    return 1;
}

if (pid == 0)
{
    printf("\nChild PID: %d\n", getpid());

    show_memory("Child before modification:");

    printf("Child modifying data...\n");

    for (int i = 0; i < SIZE; i++)
        data[i] = 'B';

    show_memory("Child after modification:");

    printf("Child data[0] = %c\n", data[0]);

    free(data);
    exit(0);
}
else
{
    wait(NULL);

    show_memory("\nParent after child modification:");

    printf("Parent data[0] = %c\n", data[0]);

    free(data);
}

return 0;
}
```

Output:

```
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ nano cow.c
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ gcc cow.c -o cow
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ ./cow
Parent PID: 1399
Parent before fork: VmRSS:          11856 kB

Child PID: 1400
Child before modification: VmRSS:          11248 kB
Child modifying data...
Child after modification: VmRSS:          11248 kB
Child data[0] = B

Parent after child modification: VmRSS:      11984 kB
Parent data[0] = A
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ |
```

Observation:

I learned how Copy-on-Write works after a fork(). I understood that the parent and child initially share memory pages, and when the child modifies the data, Linux creates a separate copy for the child. I also learned how to observe process memory usage using /proc and process monitoring commands